

Advanced Algorithms and Data Structures

Lecture# 08: Dynamic Programming

Introduction

- **Dynamic Programming (DP)** is a general algorithm design technique for solving problems defined by recurrences with overlapping subproblems.
- It was invented by American mathematician **Richard Bellman** in the 1950s to solve optimization problems and later assimilated by CS
- “Programming” here means “planning”, not “coding”.
- It is a powerful design technique that can be used to solve problems of a computational nature
- It is typically applied to **optimization** problems
- Note: Programming in this term refers to a tabular method.

How does it work?

- **Identify Subproblems:** Divide the main problem into smaller, independent subproblems.
- **Store Solutions:** Solve each subproblem and store the solution in a table or array.
- **Build Up Solutions:** Use the stored solutions to build up the solution to the main problem.
- **Avoid Redundancy:** By storing solutions, DP ensures that each subproblem is solved only once, reducing computation time.

DP vs DC

- Divide-and-conquer algorithms partition the problem into disjoint subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.
- In contrast, dynamic programming applies when the subproblems overlap (when subproblems share subsubproblems).
- A dynamic-programming algorithm solves each subsubproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subsubproblem.

Fibonacci Numbers

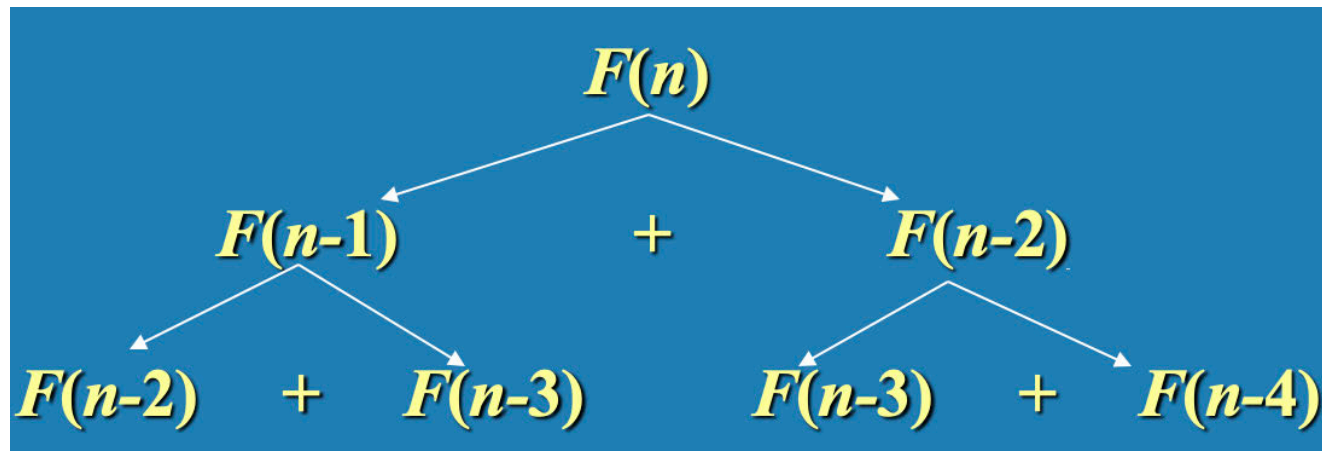
- Recall definition of Fibonacci numbers:

$$F(n) = F(n-1) + F(n-2)$$

$$F(0) = 0$$

$$F(1) = 1$$

- Computing the n^{th} Fibonacci number recursively (top-down):



Fibonacci Numbers

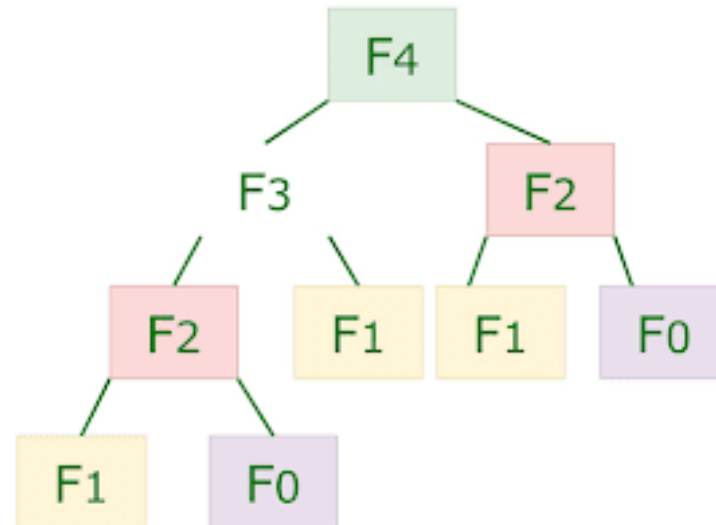
- Computing the n th Fibonacci number using bottom-up iteration and recording results:
 - $F(0) = 0$
 - $F(1) = 1$
 - $F(2) = 1+0 = 1$
 - ...
 - $F(n-2) =$
 - $F(n-1) =$
 - $F(n) = F(n-1) + F(n-2)$

0 1 1 . . . $F(n-2)$ $F(n-1)$ $F(n)$

Fibonacci Numbers

N^{th} Term of Fibonacci Series

Here F_n denotes N^{th} Term of the Fibonacci



Here Same colours denotes overlapping subproblems



Fibonacci Numbers

Fibonacci Series using Dynamic Programming

- **Subproblems:** $F(0)$, $F(1)$, $F(2)$, $F(3)$, ...
- **Store Solutions:** Create a table to store the values of $F(n)$ as they are calculated.
- **Build Up Solutions:** For $F(n)$, look up $F(n-1)$ and $F(n-2)$ in the table and add them.
- **Avoid Redundancy:** The table ensures that each subproblem is solved only once.
- By using DP, we can efficiently calculate the Fibonacci sequence without having to recompute subproblems.

When to Use DP?

- Optimal Substructure
- Overlapping subproblems

Optimal Substructure

- Show that a solution to a problem consists of making a choice, which leaves one or more subproblems to solve.
- Suppose that you are given this last choice that leads to an optimal solution.
- Given this choice, determine which subproblems arise and how to characterize the resulting space of subproblems.
- Show that the solutions to the subproblems used within the optimal solution must themselves be optimal.
Usually use cut-and-paste.
- Need to ensure that a wide enough range of choices and subproblems are considered.

Optimal Substructure

- Optimal substructure varies across problem domains:
 - 1. *How many subproblems* are used in an optimal solution.
 - 2. *How many choices* in determining which subproblem(s) to use.
- Informally, running time depends on (# of subproblems overall) $\times$ (# of choices).
- *How many subproblems and choices do the examples considered contain?*
- Dynamic programming uses optimal substructure **bottom up**.
 - *First* find optimal solutions to subproblems.
 - *Then* choose which to use in optimal solution to the problem.

Overlapping subproblems

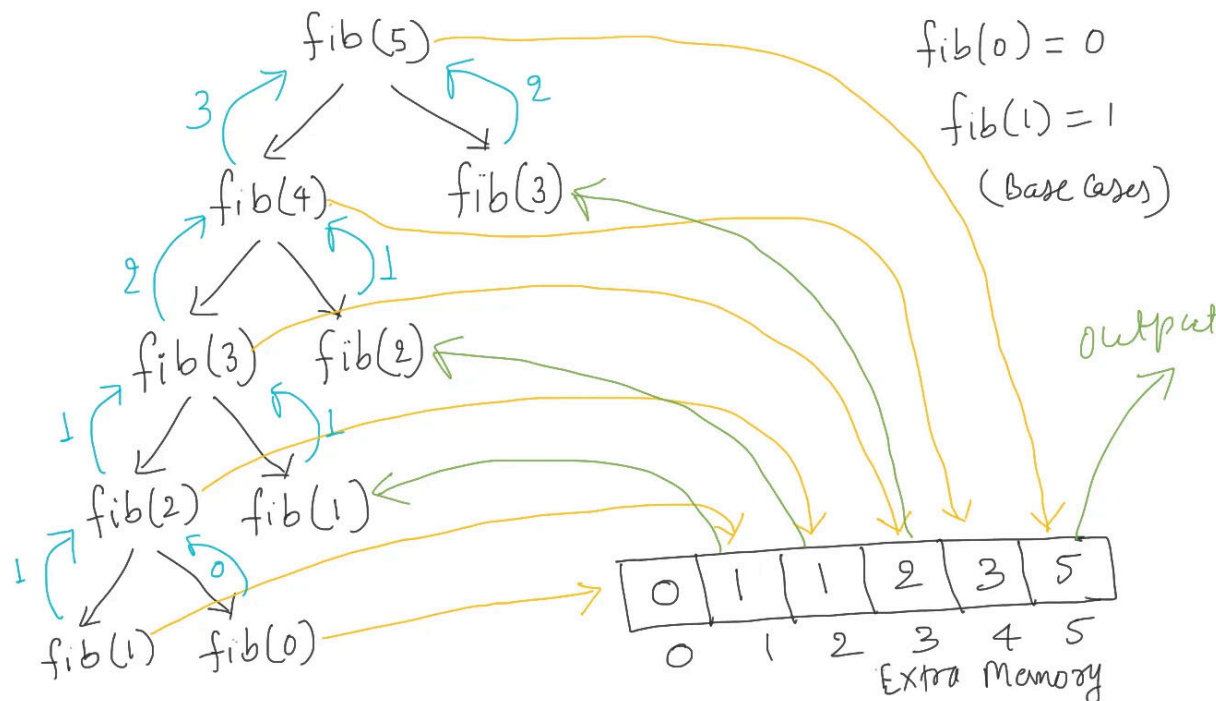
- The space of subproblems must be “small”.
- The **total number of distinct subproblems is a polynomial in the input size.**
 - A recursive algorithm is exponential because it solves the same problems repeatedly.
 - If divide-and-conquer is applicable, then each problem solved will be brand new.

Approaches of DP

- Top-Down Approach (Memoization)
- Bottom-Up Approach (Tabulation)

Memoization

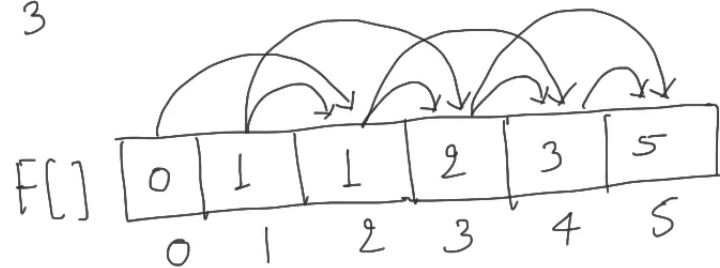
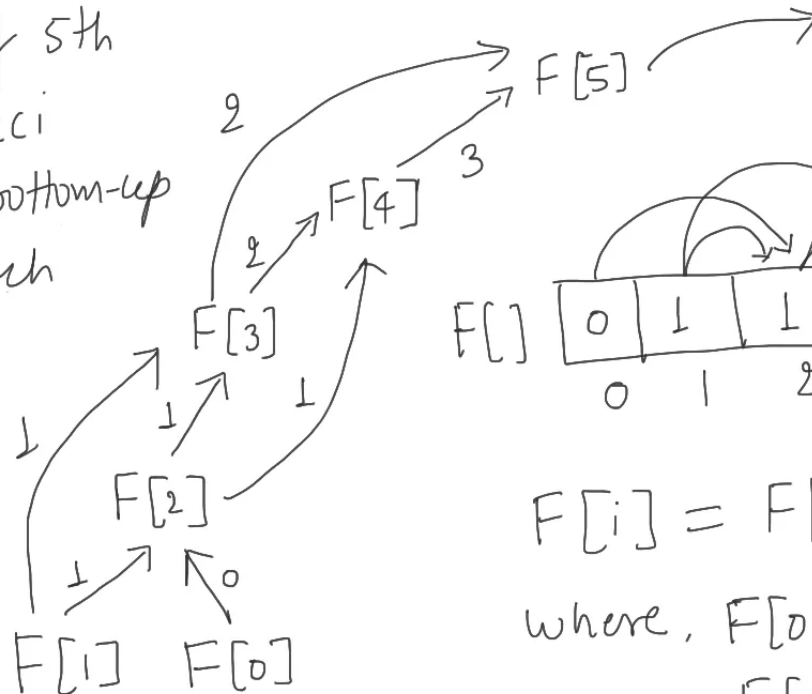
- Start with the final solution and recursively break it down into smaller subproblems. To avoid redundant calculations, we store the results of solved subproblems in a memoization table.
- Example: Finding the 5th Fibonacci using top-down approach



Tabulation

- Start with the smallest subproblems and gradually build up to the final solution. We store the results of solved subproblems in a table to avoid redundant calculations.

Calculating 5th
Fibonacci
using bottom-up
approach



$$F[i] = F[i-1] + F[i-2]$$

where, $F[0] = 0$

$F[1] = 1$

enjoyalgorithms

Advantages of DP

- Avoids recomputing the same subproblems multiple times, leading to significant time savings.
- Ensures that the optimal solution is found by considering all possible combinations.
- Breaks down complex problems into smaller, more manageable subproblems.

Applications of DP

- **Optimization:** Knapsack problem, shortest path problem, maximum subarray problem
- **Computer Science:** Longest common subsequence, edit distance, string matching
- **Operations Research:** Inventory management, scheduling, resource allocation